



**ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA**

**Laurea in Ingegneria Informatica
ALLEGATO - Abstract e Indice in Italiano**

**Exploring Phase-Change Memory as a Compute Accelerator Module:
A Virtual Prototype on the PULP Platform**

**Relatore:
Prof.
Giuseppe Tagliavini**

**Presentata da:
Leonardo Domenicali**

**Invernale
2024/2025**

Exploring Phase-Change Memory as a Compute Accelerator Module:

A Virtual Prototype on the PULP Platform

Leonardo Domenicali

Abstract

Le architetture di Von Neumann stanno raggiungendo il limite. La separazione fisica tra memoria e unità di elaborazione crea un collo di bottiglia che sta diventando critico per le applicazioni basate sull'elaborazione di grandi quantità di dati, in particolare nell'ambito dell'Intelligenza Artificiale e del Machine Learning. Lo spostamento continuo dei dati tra CPU e memoria sta diventando più dispendioso in termini di tempo ed energia rispetto alla computazione stessa: questo è il problema noto come *"memory wall"*.

Questa tesi esplora la memoria a cambiamento di fase (PCM, Phase-Change Memory) come potenziale soluzione attraverso il paradigma del calcolo analogico in memoria (AIMC, Analog In-Memory Computing). L'idea fondamentale è quella di eseguire le operazioni di calcolo direttamente dove i dati sono memorizzati, invece di affidarsi a costosi cicli di trasferimento. La PCM risulta particolarmente interessante per questo tipo di applicazioni poiché i suoi stati di resistenza analogici possono rappresentare in modo naturale i pesi di una rete neurale, e la sua architettura a matrice (crossbar) permette l'esecuzione parallela di moltiplicazioni matrice-vettore in un'unica operazione analogica.

Il principale contributo di questo lavoro è la realizzazione di un prototipo virtuale funzionante di un acceleratore AIMC basato su PCM. Il modello è implementato in C++ e integrato nella piattaforma di simulazione GVSoc, all'interno dell'ecosistema PULP. Una parte significativa del lavoro si è concentrata sull'ottimizzazione della simulazione, rendendola sufficientemente veloce da essere utile per esperimenti pratici. Ciò ha comportato la progettazione di strutture dati ottimizzate per la cache e l'implementazione di esecuzione multithread per l'algoritmo principale di moltiplicazione matrice-vettore. I test prestazionali dimostrano l'efficacia di queste ottimizzazioni: la struttura dati a buffer lineare riduce gli accessi in lettura di $5\times$ rispetto all'approccio tradizionale basato su array multidimensionali, mentre il multithreading offre un notevole incremento di velocità sui sistemi multicore, con prestazioni ottimali ottenute con 8 thread nelle configurazioni testate.

Il risultato è un framework di simulazione configurabile, utilizzabile per esplorare architetture AIMC e fornire una base per la co-progettazione di acceleratori hardware e del software necessario al loro utilizzo efficace.

Chapter 1

Introduzione

(pagina 1)

1.1 Il Muro della Memoria e l'architettura Von Neumann

(pagina 1)

1.2 Obbiettivi e motivazioni della tesi

(pagina 1)

1.3 Sommario dei contributi

(pagina 3)

Chapter 2

Contesto e Tecnologie Abilitanti

(pagina 5)

2.1 Memorie a Cambiamento di Fase (PCM)

(pagina 6)

2.2 Computazione Analogica in Memoria (AIMC)

(pagina 7)

2.3 Applicazione dell'AIMC alle PCM

(pagina 7)

2.3.1 Architettura del Modulo PCM

(pagina 7)

2.4 Piattaforma PULP

(pagina 8)

2.4.1 Panoramica PULP

(pagina 8)

2.4.2 Simulatore GVSoC

(pagina 9)

Chapter 3

Implementazione Modulo PCM

(pagina 10)

3.1 Integrazione con GVSoC

(pagina 10)

3.1.1 Struttura del Modulo

(pagina 10)

3.1.2 Interfaccia dei Bus

(pagina 11)

3.1.3 Modello di Sincronizzazione

(pagina 11)

3.1.4 Gestione Energetica

(pagina 12)

3.2 Vincoli Architettureali

(pagina 13)

3.2.1 Vincoli Hardware

(pagina 13)

3.2.2 Vincoli Software

(pagina 13)

3.3 Architettura del Modulo PCM

(pagina 13)

3.4 Struttura Dati del Modulo

(pagina 13)

3.4.1 Ottimizzazione Struttura Dati

(pagina 14)

Vettore Quadridimensionale

(pagina 14)

Buffer Lineare

(pagina 15)

3.4.2 Algoritmo Differenziale

(pagina 15)

3.5 Algoritmo di Moltiplicazione Matrice-Vettore (MVM)

(pagina 16)

3.5.1 Implementazione di Base

(pagina 16)

3.6 Ottimizzazione

(pagina 16)

3.6.1 Ottimizzazione per Cache

(pagina 16)

3.6.2 Riutilizzo dei Registri

(pagina 17)

3.6.3 Esecuzione Multi-Threaded

(pagina 17)

3.7 Conversione Analogico-Digitale e Digitale-Analogico

(pagina 18)

3.7.1 Conversione Digitale-Analogico

(pagina 18)

3.7.2 Conversione Analogico-Digitale

(pagina 19)

3.8 Mappatura in Memoria e Configurazione

(pagina 19)

Chapter 4

Test e Risultati

(pagina 21)

4.1 Analisi e Test dell'Algoritmo

(pagina 21)

4.1.1 Valutazione Dati

(pagina 21)

Metriche di Prestazione

(pagina 21)

4.1.2 Analisi delle Performance

(pagina 24)

Impatto delle Ottimizzazioni del Compilatore

(pagina 24)

Comportamento delle Cache

(pagina 24)

Pattern di Accesso alla Memoria

(pagina 25)

Performance del Multithreading

(pagina 25)

Analisi del Codice Generato

(pagina 25)

4.2 Validazione del Modulo

(pagina 26)

4.2.1 Validazione del Algoritmo

(pagina 26)

4.2.2 Validazione del Integrazione su GVSoC

(pagina 26)

Chapter 5

Conclusione e Lavori Futuri

(pagina 28)

5.1 Conclusioni

(pagina 28)

5.2 Lavori Futuri

(pagina 28)